Mode Discovery via Kernel Density Estimation

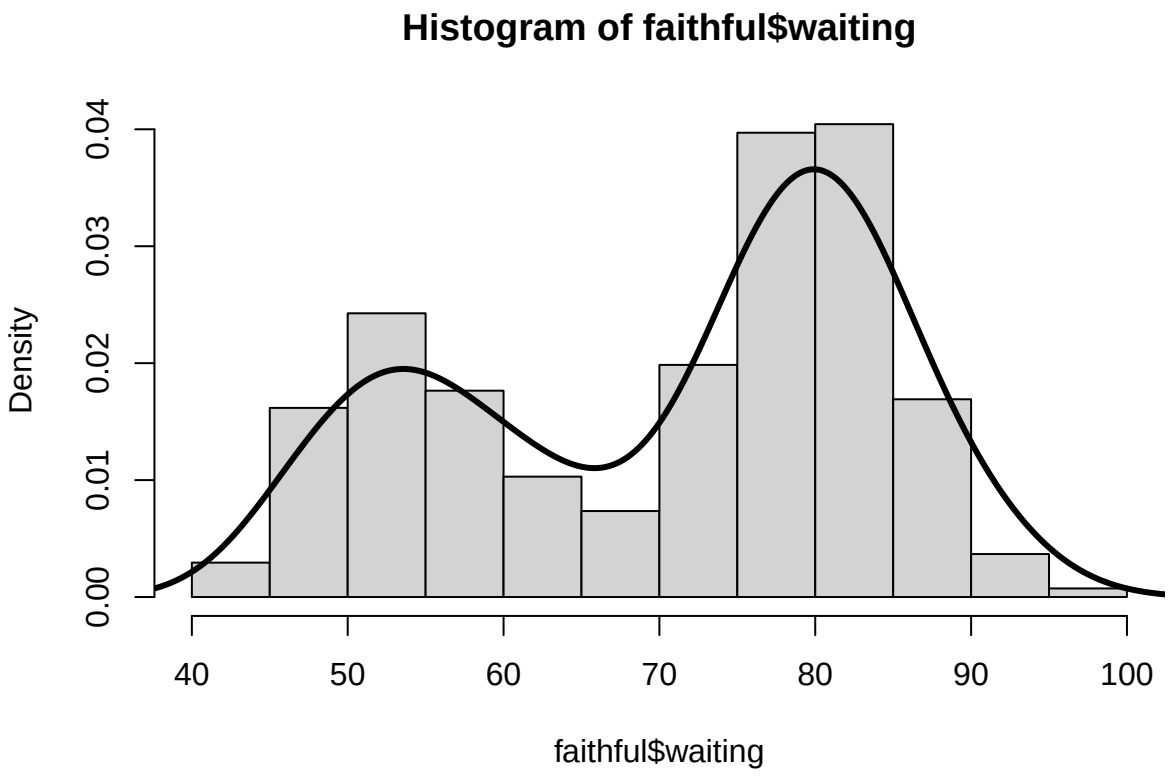
Instead of finding the minimum loss, we can also use gradient descent to find the mode of a continuous p.d.f. $f(x)$ by finding either

$$\arg\min_{x \in \Re} -f(x)$$

or Newton's method and solving for $f'(x) = 0$.

a) (2 marks) Load the Old Faithful dataset, take the kernel density estimate (KDE) with `density()`, and plot the KDE of the waiting time between eruptions with the following code

```
data("faithful")
kde = density(faithful$waiting)


hist(faithful$waiting, prob=TRUE)
lines(kde$x, kde$y, lwd=3)
```



Briefly describe the challenges that methods like gradient descent and Newton-Raphson will have with this data.

Methods like gradient descent and Newton-Raphson face significant challenges with the **Old Faithful** dataset because the distribution is **bimodal**, exhibiting two distinct peaks in the waiting time between eruptions. Since these optimization algorithms are local search methods designed to find a single optimum, they are highly sensitive to the **starting point** ($\hat{\theta}_0$). If the algorithm begins near a local mode rather than the global one, it may converge to that inferior peak and fail to explore the rest of the distribution. Additionally, Newton-Raphson specifically requires solving for $f'(x) = 0$, and in regions where the density is nearly flat or contains multiple inflection points, the method can become unstable or converge to a local minimum instead of the desired maximum.

b) (1 mark) Recreate the KDE as a function called `myKDE` by recreating what the KDE is doing:

$$\text{KDE}(\theta) = \frac{1}{n}\sum_{i=1}^{n} f(\theta, y_i, \sigma) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{(\theta - y_i)^2}{2\sigma^2}}$$

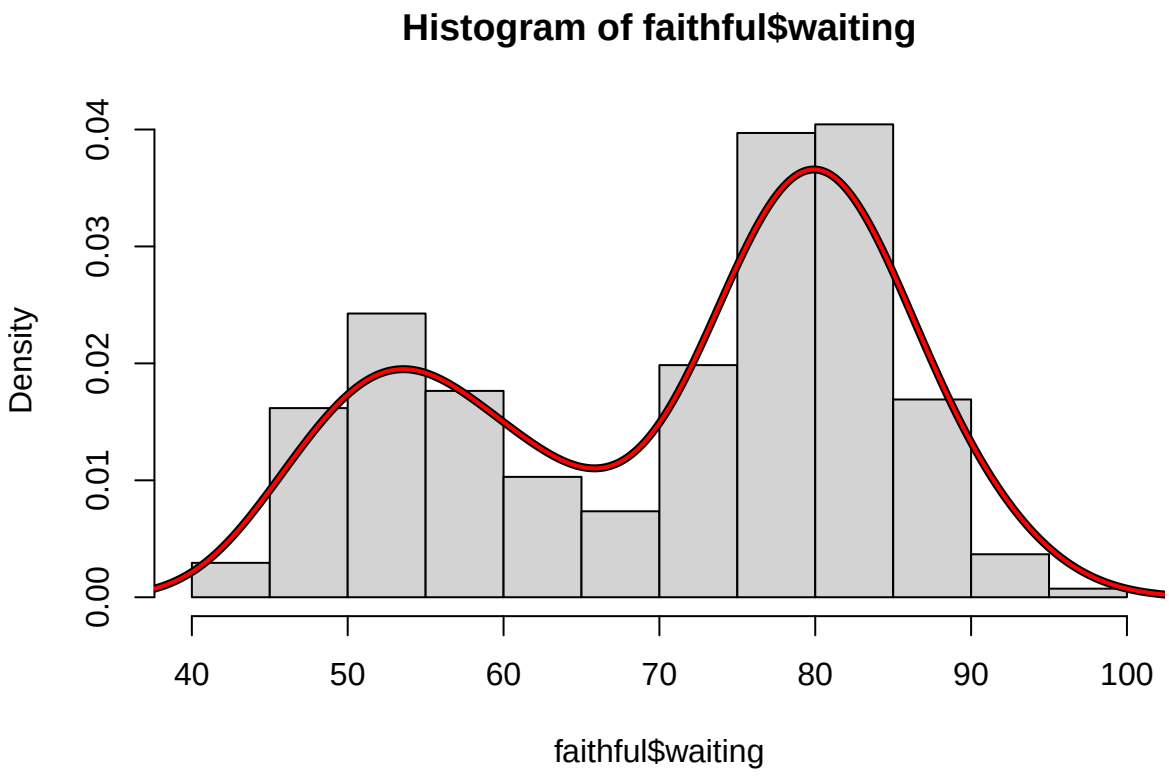
where $f(x, y_i, \sigma)$ is the p.d.f. of the normal distribution at x with mean y_i and standard deviation σ, and where n is the number of observations y_i.

Let $\sigma = 3.987559$, a value determined by the KDE function based on the data. You may use the `dnorm()` function to get the p.d.f. of the normal.

```
sigg = 3.987559

myKDE = function(theta)
{
  y = faithful$waiting
  n = length(y)

  rho = mean(dnorm(theta, mean = y, sd = sigg))
  return(rho)
}
```

Use this code to check your work. The red line curve should overlap the black curve if you've got it right.

```
myKDE_x = kde$x
myKDE_y = numeric(0)

for(i in 1:length(myKDE_x))
{
  myKDE_y[i] = myKDE(myKDE_x[i])
}
hist(faithful$waiting, prob=TRUE)
lines(kde$x, kde$y, lwd=4)

# This red line should overlap the black line if code is right
lines(myKDE_x, myKDE_y, lwd=2, col="red")
```



c) (4 marks) Use the fact that the derivative of the normal distribution is

$$\frac{df}{d\theta} = \frac{\theta}{\sigma^3\sqrt{2\pi}} e^{-x^2/2\sigma^2}$$

to make factory function for rho and gradient functions, (Hint: `rho` should just be the negative of `myKDE`)

$$\frac{d}{d\theta} KDE = \frac{1}{n}\sum_{i=1}^{n} \frac{d}{d\theta} f(\theta, y_i, \sigma) = \frac{1}{n}\sum_{i=1}^{n} \frac{\theta}{\sigma^3\sqrt{2\pi}} e^{-(\theta - y_i)^2/2\sigma^2}$$

```
createKDERho <- function(y) {
  ## local variable
  sigg = 3.987559
  n = length(y)
  ## Return this function
  function(theta) {
    KDEval <- mean(dnorm(theta, mean = y, sd = sigg))
    -KDEval
  }
}

KDErho = createKDERho(faithful$waiting)
KDErho(50)
```

```
## [1] -0.0173336
```

```
### Similarly for the gradient function
createKDEGradient <- function(y) {
  ## local variables
  sigg = 3.987559
  n = length(y)
  function(theta) {
    1/n * sum(theta/(sigg^3 * sqrt(2*pi))*(exp(1)^(-(theta-y)^2/(2*sigg^2))))
  }
}

KDEgrad = createKDEGradient(faithful$waiting)
```

d) (2 marks) Use the gradient function from part c in `gradientDescent` and find the mode from the starting points $\hat{\theta}_0 = 40$, $\hat{\theta}_0 = 60$, $\hat{\theta}_0 = 80$, $\hat{\theta}_0 = 100$. Use the output to find what the global KDE mode is.

```
gradientDescent <- function(theta = 0,
      rhoFn, gradientFn, lineSearchFn, testConvergenceFn,
      maxIterations = 100,
      tolerance = 1E-6, relative = FALSE,
      lambdaStepsize = 0.01, lambdaMax = 0.5 ) {

  converged <- FALSE
  i <- 0

  while (!converged & i <= maxIterations) {
    g <- gradientFn(theta) ## gradient
    glength <-  sqrt(sum(g^2)) ## gradient direction
    if (glength > 0) d <- g /glength

    lambda <- lineSearchFn(theta, rhoFn, d,
                lambdaStepsize = lambdaStepsize, lambdaMax = lambdaMax)

    thetaNew <- theta - lambda * d
    converged <- testConvergenceFn(thetaNew, theta,
                                   tolerance = tolerance,
                                   relative = relative)
    theta <- thetaNew
    i <- i + 1
  }

  ## Return last value and whether converged or not
  list(theta = theta, converged = converged, iteration = i, fnValue = rhoFn(theta))
}
```

```
### line searching could be done as a simple grid search
gridLineSearch <- function(theta, rhoFn, d,lambdaStepsize = 0.01, lambdaMax = 1) {
  ## grid of lambda values to search
  lambdas <- seq(from = 0, by = lambdaStepsize,  to = lambdaMax)
  ## line search
  rhoVals <- sapply(lambdas, function(lambda) {rhoFn(theta - lambda * d)})
  ## Return the lambda that gave the minimum
```

```
  lambdas[which.min(rhoVals)]
}
```

```
### Where testCovergence might be (relative or absolute)
testConvergence <- function(thetaNew, thetaOld, tolerance = 1E-10, relative=FALSE) {
   sum(abs(thetaNew - thetaOld)) < if (relative) tolerance * sum(abs(thetaOld)) else tolerance
}
```

```
gradientDescent(theta = 40,
      KDErho, KDEgrad, gridLineSearch, testConvergence,
      maxIterations = 100,
      tolerance = 1E-6, relative = FALSE,
      lambdaStepsize = 0.01, lambdaMax = 0.5 )
```

```
## $theta
## [1] 40
##
## $converged
## [1] TRUE
##
## $iteration
## [1] 1
##
## $fnValue
## [1] -0.002149866
```

```
gradientDescent(theta = 60,
      KDErho, KDEgrad, gridLineSearch, testConvergence,
      maxIterations = 100,
      tolerance = 1E-6, relative = FALSE,
      lambdaStepsize = 0.01, lambdaMax = 0.5 )
```

```
## $theta
## [1] 53.57
##
## $converged
## [1] TRUE
##
## $iteration
## [1] 14
##
## $fnValue
## [1] -0.01950572
```

```
gradientDescent(theta = 80,
      KDErho, KDEgrad, gridLineSearch, testConvergence,
      maxIterations = 100,
      tolerance = 1E-6, relative = FALSE,
      lambdaStepsize = 0.01, lambdaMax = 0.5 )
```

```
## $theta
```

```
## [1] 79.91
##
## $converged
## [1] TRUE
##
## $iteration
## [1] 2
##
## $fnValue
## [1] -0.03658561
```

```
gradientDescent(theta = 100,
      KDErho, KDEgrad, gridLineSearch, testConvergence,
      maxIterations = 100,
      tolerance = 1E-6, relative = FALSE,
      lambdaStepsize = 0.01, lambdaMax = 0.5 )
```

```
## $theta
## [1] 79.91
##
## $converged
## [1] TRUE
##
## $iteration
## [1] 42
##
## $fnValue
## [1] -0.03658561
```

The global KDE mode is 79.91.